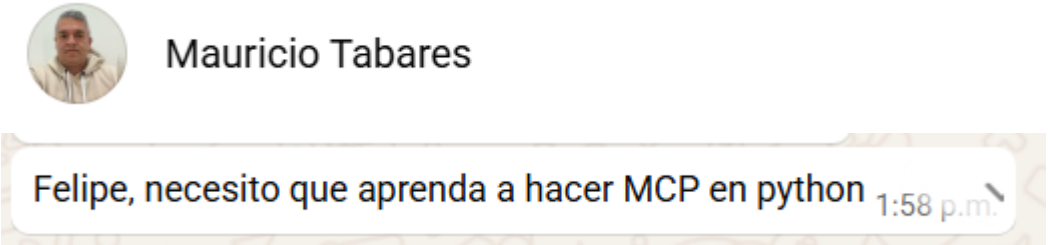
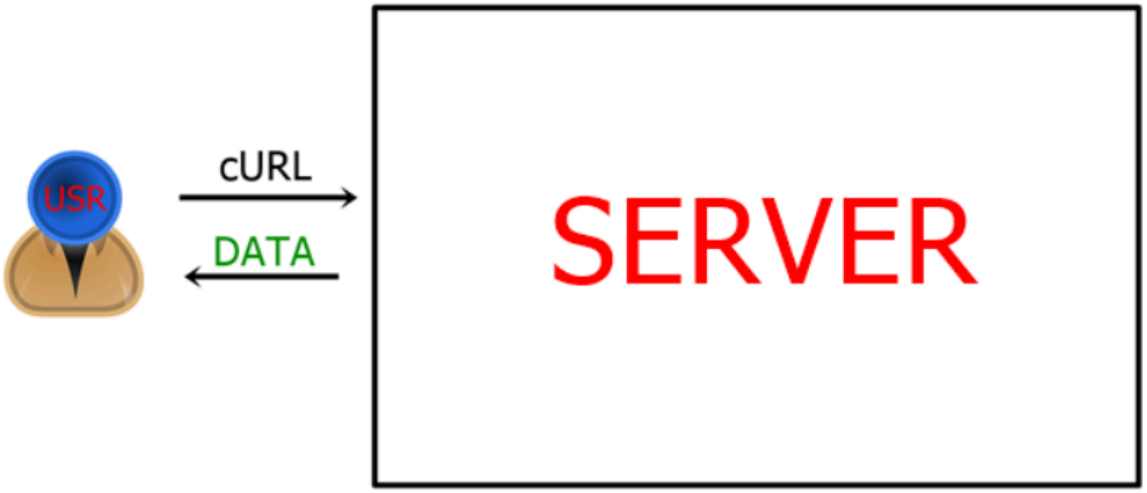
a los 26 días de 04 de 2026 se me asigna una misión:



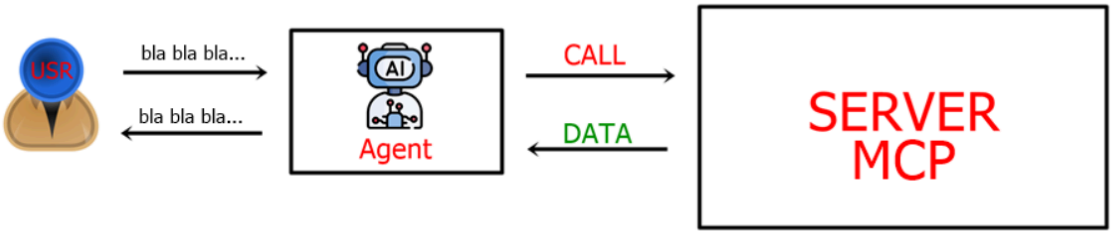

…

# *MODEL CONTEXT PROTOCOL*

Antiguamente los servidores utilizan un protocolo para comunicarse con los servidores HTTPs + REST + cURL con ello se le pueden enviar peticiones explícitas a un servidor y el servidor responde.




Ahora bien se ha diseñado un nuevo protocolo que se llama MCP en donde se pueden conectar AGENTES DE IA con fuentes de datos o herramientas externas.




La pila de llamados es la siguiente:

- Un agente recibe una instrucción en lenguaje natural.

- El agente usando MCP traduce lo que dice el usuario a llamadas estructuradas hacia lo que un servidor expone.
- El servidor recibe las instrucciones en un lenguaje del siguiente estilo:
  Ejemplo:
  ```
  read_resource("database://users")
  call_tool("calcular_iva", {"monto": 100})
  ```
- El servidor MCP ejecuta la acción y devuelve la data.
- El agente interpreta la respuesta y responde al usuario.

Por ello MCP es solo un puente entre lenguaje natural y un servidor…

Ventajas:
- Un solo agente se puede conectar a múltiples servidores.
- El agente descubre en tiempo real que nuevas herramientas tiene el servidor.
- Es independiente la IA del lenguaje de programación del servidor.
- Esto es lo que va a reemplazar a los servidores tradicionales.

Lo que el lector tiene que comprender es que la meta no es diseñar ENDPOINTs a la medida sino desarrollar interfaces que sean fáciles de comprender para una IA.

En pocas palabras tu objetivo será diseñar código “AI-comprehensible”

# *Arquitectura recomendada*

Arquitectura en capas resumida:

- Presentación: MCP.
- Lógica de negocio: Auth, MCP TOOLs.
- Repositorios: DB.

Arquitectura sería:

```
MCP_SERVER/
├── src/
│   ├── main.py                 # Punto de entrada, levanta el MCP server
│   ├── interfaces/             # Capa de presentación (MCP Tools)
│   │   ├── __init__.py
│   │   └── herramientas/       # Agrupación de tools por dominio
│   │       ├── usuario_tools.py
│   │       └── producto_tools.py
│   ├── application/            # Casos de uso (lógica de negocio pura)
│   │   ├── __init__.py
│   │   ├── services/           # Servicios de aplicación
│   │   │   ├── usuario_service.py
│   │   │   └── producto_service.py
│   │   └── dto/                # Objetos de transferencia (entrada/salida)
│   │       └── schemas.py
│   ├── domain/                 # Reglas del negocio (independiente de frameworks)
│   │   ├── __init__.py
│   │   ├── entities/           # Entidades de dominio
│   │   │   └── usuario.py
│   │   ├── repositories/       # Contratos (interfaces) de repositorios
│   │   │   └── usuario_repo.py
│   │   └── exceptions/         # Excepciones de negocio
│   │       └── domain_errors.py
│   ├── infrastructure/         # Detalles concretos (DB, APIs, etc.)
│   │   ├── __init__.py
│   │   ├── db/                 # Implementaciones de repositorios
│   │   │   ├── connection.py   # Pool de conexiones, configuración
│   │   │   └── usuario_repo_impl.py
│   │   └── di/                 # Configuración de inyección de dependencias
│   │       └── container.py    # Assembler de dependencias
│   └── config/                 # Configuración (variables de entorno, etc.)
│       ├── __init__.py
│       └── settings.py
├── tests/                      # Tests unitarios / integración
│   ├── test_usuario_tools.py
│   ├── test_usuario_service.py
│   └── conftest.py
├── requirements.txt
└── pyproject.toml
```

| ***Nr*** | ***Nombre*** | ***Descripción*** |
|---|---|---|
| 01 | herramientas | Traduce las peticiones de CLAUDE en casos de uso.<br><br>Nota: tiene que ser super delgada, valida solo tipos de datos, capturar excepciones y devolver respuestas legibles para la IA. |
| 02 | Casos de uso. | Orquesta la lógica de negocio. Usa repositorios a través de interfaces.<br><br>Nota: Son totalmente independientes de MCP y los TEST son con MOCK. |
| 03 | Entidades | Son los objetos de dominio. |
| 04 | Repositorios Interfaces | Son los contratos. |
| 05 | Infraestructura | Implementaciones de los contratos. |

| | | |
|---|---|---|
| 06 | DY | Inyección de dependencias. |
| 07 | config | .env |
| | | |

# *Ejemplos de código*

Ejemplo TOOL:

```
@mcp.tool()
def crear_usuario(nombre: str, email: str) -> dict:
    try:
        dto = CrearUsuarioDTO(nombre=nombre, email=email)
        resultado = usuario_service.crear(dto)
        return {"exito": True, "id": resultado.id}
    except EmailInvalidoError as e:
        return {"error": str(e)}
```

Ejemplo USE_CASE

```
class UsuarioService:
    def __init__(self, repo: UsuarioRepository, hash_service: HashService):
        self.repo = repo
        self.hash_service = hash_service

    def crear(self, dto: CrearUsuarioDTO) -> UsuarioDTO:
        if self.repo.existe_email(dto.email):
            raise EmailYaRegistradoError()
        hashed = self.hash_service.hash(dto.password)
        usuario = Usuario(nombre=dto.nombre, email=dto.email, password_hash=hashed)
        self.repo.guardar(usuario)
        return UsuarioDTO.from_entity(usuario)
```

Ejemplo DY

```python
# container.py
from src.infrastructure.db.usuario_repo_impl import UsuarioRepoImpl
from src.application.services.usuario_service import UsuarioService

def build_usuario_service(db_config) -> UsuarioService:
    repo = UsuarioRepoImpl(db_config)
    hash_svc = SHA256HashService()  # otra impl
    return UsuarioService(repo, hash_svc)
```

Advertencias: Las TOOLS no pueden instanciarse… deben de ser inyectables ejemplo desde main:

```python
# src/main.py
container = build_container(config)
usuario_svc = container.usuario_service
register_usuario_tools(mcp, usuario_svc)
```

# *LIVE DEMO QUE MONTAREMOS EN LOCAL*

Advertencia: esto solo es un ejemplo rápido para ver su funcionamiento en local.

# *Sistema de eventos para Rappi – Integración con Claude Desktop vía MCP*

***Objetivo general***
Montar un prototipo funcional que permita gestionar usuarios, eventos, acumulaciones y redenciones (típicos en programas de lealtad o campañas) desde Claude Desktop, utilizando una base de datos PostgreSQL dockerizada y un servidor MCP escrito en Python.

Elementos del demo

| ***Componente*** | ***Tecnología*** | ***Descripción*** |
|---|---|---|

| | | |
|---|---|---|
| Base de datos | PostgreSQL 16 | Almacena las tablas: usuarios, eventos, evento_acumulacion, evento_redencion. Los datos persisten gracias a un volumen Docker. |
| Servidor MCP | FastMCP + psycopg2 | Expone “herramientas” (tools) que Claude puede invocar. Cada herramienta ejecuta operaciones en la BD (insertar, consultar). Corre dentro de otro contenedor Docker. |
| Conector a BD | psycopg2 | Permite al servidor Python comunicarse con PostgreSQL de forma eficiente y segura. |
| Tipado | Python typing + pydantic | Garantiza claridad en el código, autocompletado y detección temprana de errores (muy útil para mantenimiento futuro). |
| Orquestación | Docker Compose | Levanta los dos contenedores (PostgreSQL y servidor MCP) con una sola orden, gestiona redes y volúmenes. |
| Interfaz | Claude Desktop (Windows 11) | Conexión vía SSE (Server-Sent Events) al servidor MCP local. Configuración en Settings > Developer > MCP Servers. Permite usar lenguaje natural para manejar los datos. |

Arquitectura simple:

```
rappi-mcp/
├── docker-compose.yml
├── .env (opcional, para variables)
├── postgres/
│   └── init.sql
├── mcp-server/
│   ├── Dockerfile
│   ├── pyproject.toml
│   ├── main.py
│   └── .env (opcional)
└── README.md
```

# *Creación de la red de trabajo de proyectos*

Vamos a crear una red de docker:

docker network create redDemoMcp

```
C:\WINDOWS\system32\cmd.

Microsoft Windows [Versión 10.0.26200.8246]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\docto>docker network create redDemoMcp
6843540b72d118bde6241420eb9bdcb0601977efc2545dfed7456b5db3f047bf
```

# *Creación de la base de datos*

Vamos a crear una imagen de postgres dockerizada

docker run -d --name postgres_mcp_demo --network redDemoMcp -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=admin123 -e POSTGRES_DB=demomcp -p 5432:5432 postgres:16

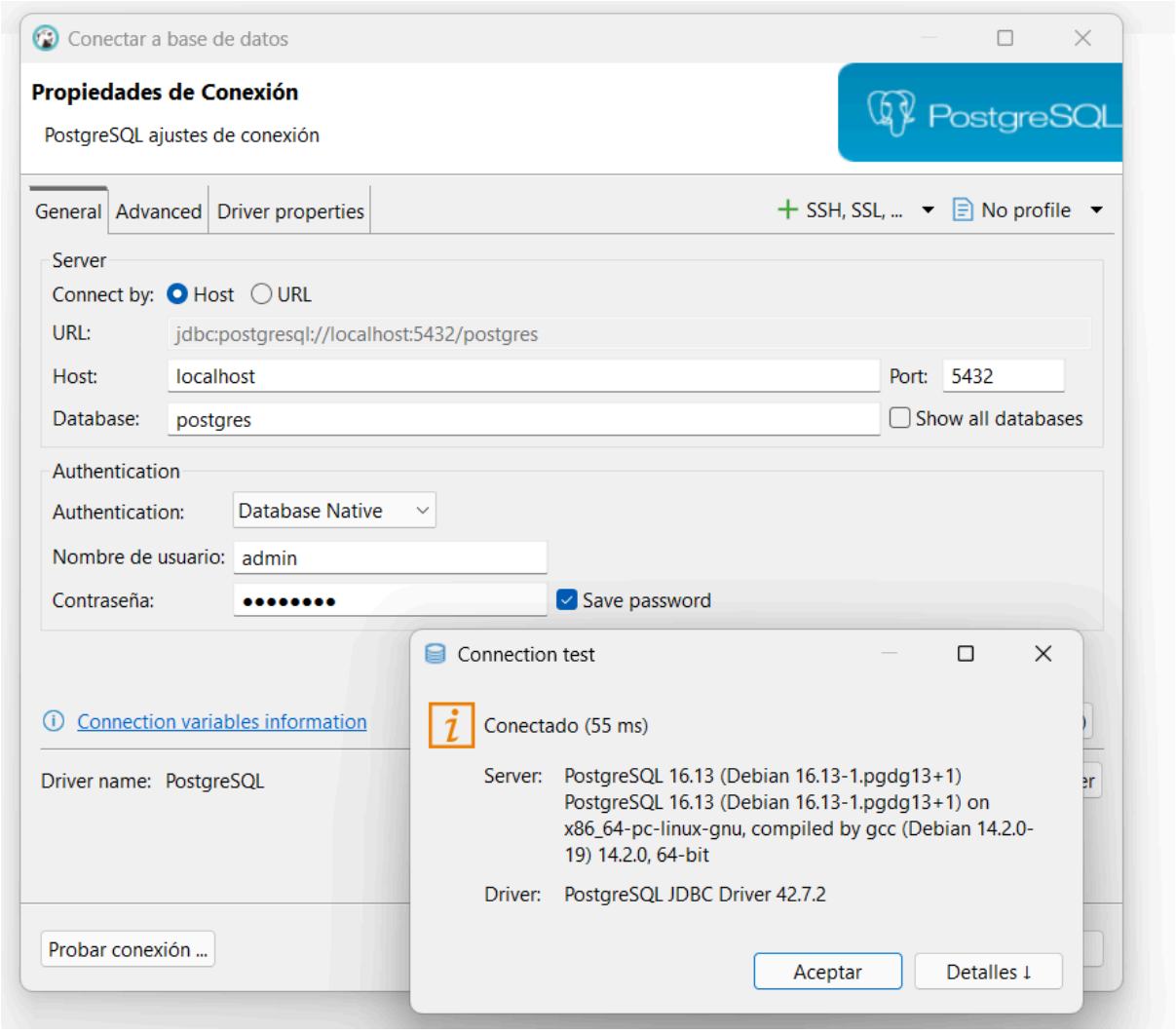
```
C:\WINDOWS\system32\cmd.

C:\Users\docto>docker run -d --name postgres_mcp_demo --network redDemoMcp -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=admin123 -e POSTGRES_DB=demomcp -p 54
32:5432 postgres:16
Unable to find image 'postgres:16' locally
16: Pulling from library/postgres
3531af2bc2a9: Extracting 1 s
decd178ddeb0: Download complete
a88738cb3441: Download complete
9ebf6f2e52f3: Download complete
cef33b0ec231: Download complete
d65fa596db1b: Download complete
f2aef946c609: Download complete
102496efe6c1: Download complete
2952c21578f5: Download complete
3326b6522e5b: Download complete
9e42eccf4bc0: Download complete
a13375e48b67: Download complete
8e80c373fbad: Downloading [>                                        ]  1.049MB/113.1MB
3618b5f7b5bc: Download complete
```

…

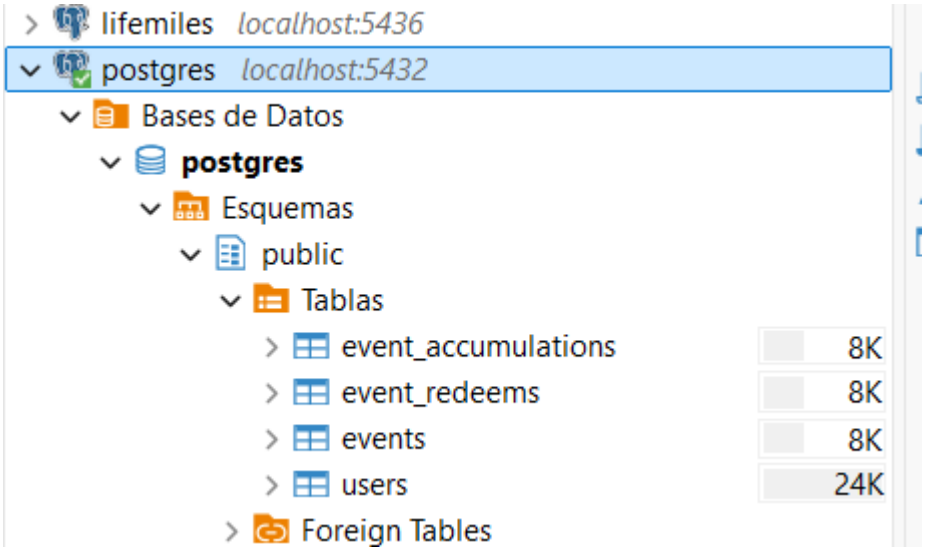
| | | | | | | |
|---|---|---|---|---|---|---|
| ☐ | ● | postgres_mcp_demo | 3d221563c1a8 | postgres:1( | 5432:5432 | 0.04% 29.63MB / 7.6GB |

Creación de las tablas mediante conector de base de datos DBeaver




```sql
-- Tabla users
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  rappi_id VARCHAR(100),
  external_id VARCHAR(100),
  name VARCHAR(200) NOT NULL,
  email VARCHAR(200) UNIQUE NOT NULL,
  phone VARCHAR(50)
);
-- Tabla events
CREATE TABLE events (
  id SERIAL PRIMARY KEY,
  event_id VARCHAR(100) NOT NULL,
  status VARCHAR(10) NOT NULL CHECK (status IN ('OK', 'FAIL')),
  points INTEGER NOT NULL DEFAULT 0,
  date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
-- Tabla event_accumulations
CREATE TABLE event_accumulations (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  points INTEGER NOT NULL DEFAULT 0
);
-- Tabla event_redeems
CREATE TABLE event_redeems (
  id SERIAL PRIMARY KEY,
```

```
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  points INTEGER NOT NULL DEFAULT 0
);
```




…

# *Creación de nuestro proyecto SERVER MCP*

Por ahora solo vamos a explicar el hola mundo… para el demo completo visita el link de github:

https://github.com/felipedelosh/mcp-server-python-demo

vamos a crear una carpeta con 3 archivos:

Principal: mcp-server\main.py

es un script muy simple que va a saludar al usuario:

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("RappiDemo")

@mcp.tool()
def saludar(nombre: str = "mundo") -> str:
    """Saluda a una persona."""
    return f"¡Hola, {nombre}!"

if __name__ == "__main__":
    # Transporte STDIO: permite a Claude Desktop hablar con este proceso
    mcp.run(transport="stdio")
```

Dependencias: mcp-server\pyproject.toml

```
mcp-server > pyproject.toml
[project]
name = "rappi-mcp-server"
version = "0.1.0"
description = "Servidor MCP simple"
requires-python = ">=3.10"
dependencies = [
    "mcp[cli]>=1.6.0",
]
```

Dockerfile: mcp-server\Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY mcp-server/pyproject.toml .
COPY mcp-server/main.py .

RUN pip install --no-cache-dir .

CMD ["python", "main.py"]
```

Y luego vamos a construir un docker-compose simple para levantar:

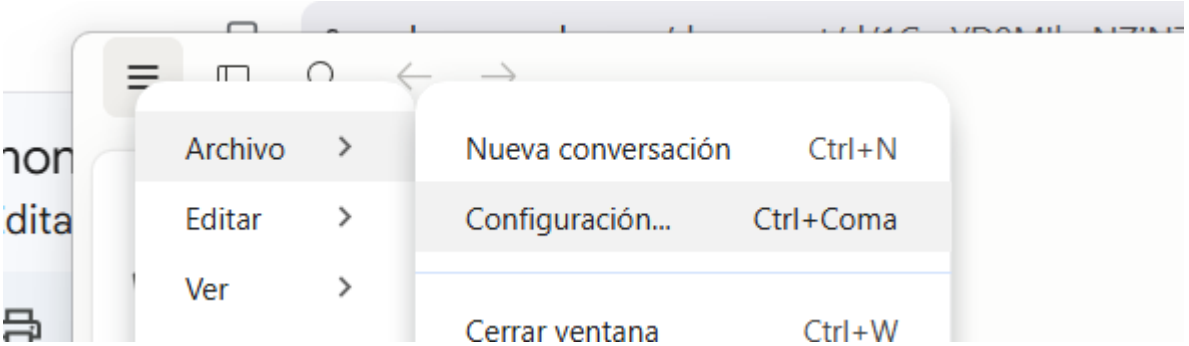
```yaml
docker-compose.yml
services:
  mcp-server:
    build:
      context: .
      dockerfile: mcp-server/Dockerfile
    container_name: rappi_mcp_stdio
    stdin_open: true
    tty: true
```

Para verificar que funciona vamos a levantar la imagen:
docker-compose up –build

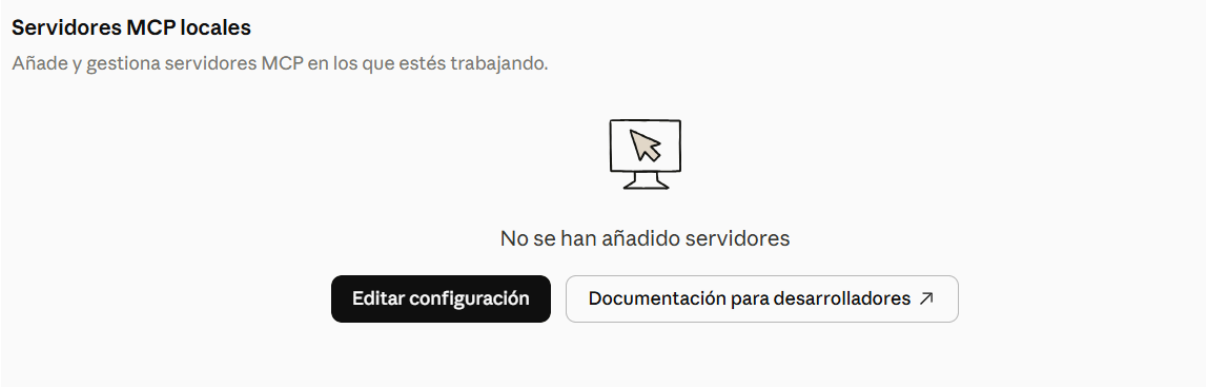
```
PS C:\Users\docto\OneDrive\Documentos\TESTIANDO\Tarea 52 - TESTIANDO - MCP Server Python> docker-compose up --build
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:6d85378d88a19cd4d76079817532d62232be95757cb45945a99fec8e80  0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:6d85378d88a19cd4d76079817532d62232be95757cb45945a99fec8e80  0.0s
 => [internal] load build context                                                                                    0.0s
 => => transferring context: 115B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                        0.0s
 => CACHED [3/5] COPY mcp-server/pyproject.toml .                                                                    0.0s
 => CACHED [4/5] COPY mcp-server/main.py .                                                                           0.0s
 => CACHED [5/5] RUN pip install --no-cache-dir .                                                                    0.0s
 => exporting to image                                                                                               0.1s
 => => exporting layers                                                                                              0.0s
 => => exporting manifest sha256:e73ae63b4b23919328dc6bfcf89875677d76bbeeba61d9cde20b89d27550033e                    0.0s
 => => exporting config sha256:30a3f44c79ea46c5efdf3e77a8b443f10b93872534cbbc7475895b0a169a0074                      0.0s
 => => exporting attestation manifest sha256:94fea0c3fbe2c766dcc6bce9a431555e170147a8301b4879c0f81f60abf6476d        0.0s
 => => exporting manifest list sha256:f40e8561cfb482fc34d8e0f05b1f1359b423cbad23ccc524f51f2034e8ce973c               0.0s
 => => naming to docker.io/library/tarea52-testiando-mcpserverpython-mcp-server:latest                              0.0s
 => => unpacking to docker.io/library/tarea52-testiando-mcpserverpython-mcp-server:latest                           0.0s
 => resolving provenance for metadata file                                                                           0.0s
[+] Running 2/2
 ✓ tarea52-testiando-mcpserverpython-mcp-server  Built                                                               0.0s
 ✓ Container rappi_mcp_stdio                     Recreated                                                           1.4s
Attaching to rappi_mcp_stdio
```

# *Conectar CLAUDE DESKTOP con nuestro proyecto*

Incluir el archivo de interconexión FILE > SETTINGS > DEVELOPER



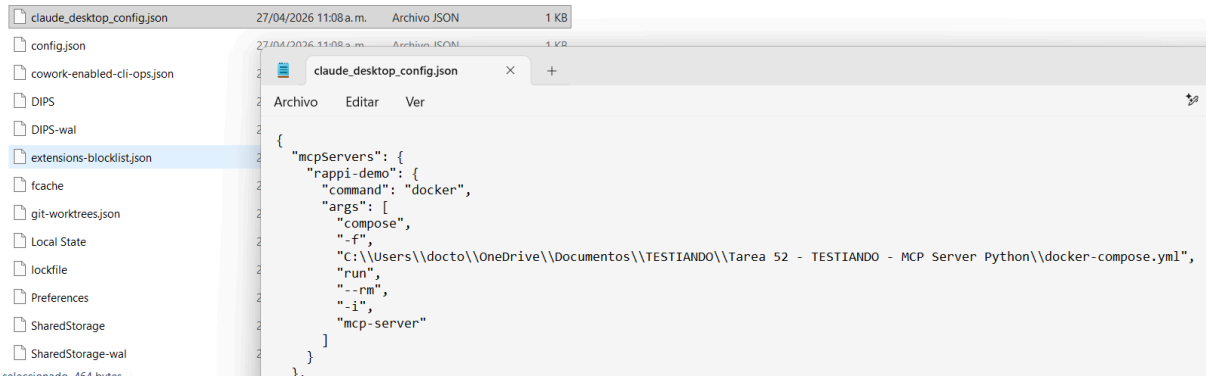

…



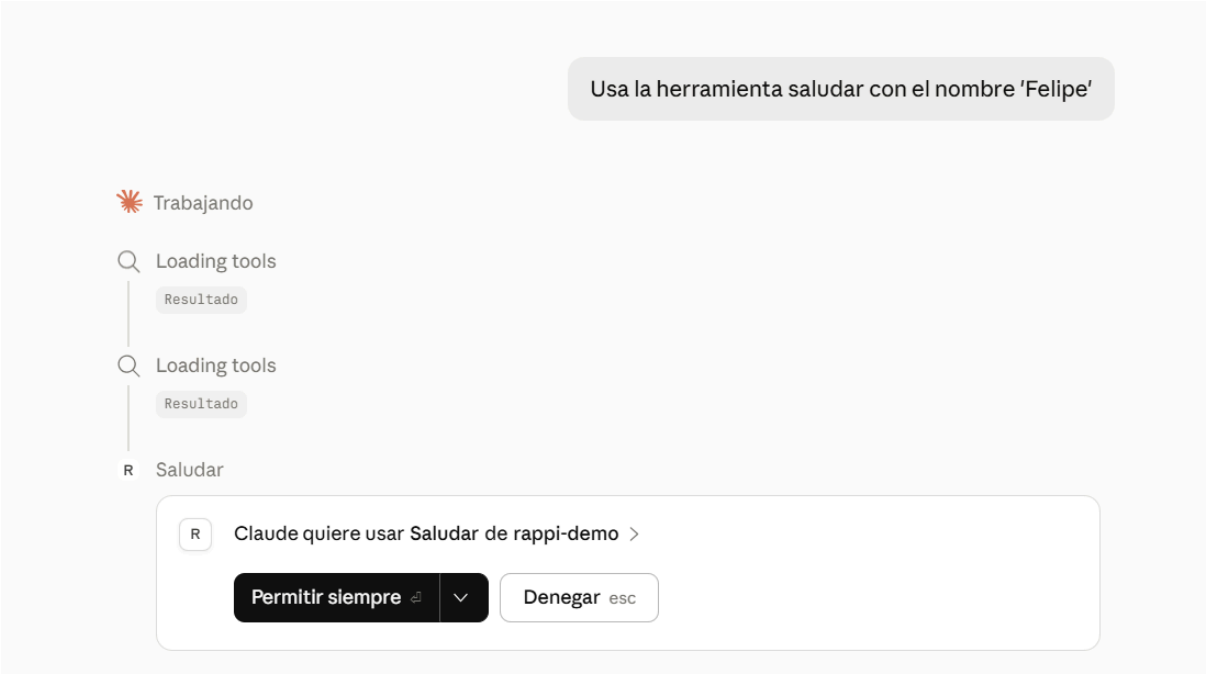

…

Y vamos a editar la configuración y luego te pedirá que edites el archivo “claude_desktop_config.json” vamos a borrar todo lo que hay ahí para agregar las siguientes líneas:

```
{
  "mcpServers": {
    "rappi-demo": {
      "command": "docker",
      "args": [
        "compose",
        "-f",
        "C:\\Users\\docto\\OneDrive\\Documentos\\TESTIANDO\\Tarea 52 - TESTIANDO - MCP Server
Python\\docker-compose.yml",
        "run",
        "--rm",
        "-i",
        "mcp-server"
      ]
    }
  }
}
```




Cierra y vuelve a abrir CLAUDE DESKTOP desde el administrador de tareas… o reinicia la computadora y pregunta:




Y luego de aceptar la herramienta funciona de la siguiente manera:

## *Recursos de aprendizaje:*

https://es.wikipedia.org/wiki/Protocolo_de_Contexto_de_Modelo

https://www.youtube.com/watch?v=xGj8Y07bGPc